

第十六课预习报告

subst.c

- 作用: 演示宏参数字符串化(#)的用法, 将宏参数转换为字符串并在输出中使用。
- 功能: 定义 `PSQR(x)` 宏, 打印参数及其平方值, 支持表达式作为参数。
- 演示效果:

```
1  /* subst.c -- substitute in string */
2  #include <stdio.h>
3  #define PSQR(x) printf("The square of " #x " is %d.\n",((x)*(x)))
4
5  int main(void)
6  {
7      int y = 5;
8
9      PSQR(y);
10     PSQR(2 + 4);
11
12     return 0;
13 }
```

```
1  The square of y is 25.
2  The square of 2 + 4 is 36.
```

useheader.c

- 作用: 演示跨文件结构体与函数的使用, 通过 `names_st.h` 和 `names_st.c` 提供姓名输入与输出功能。
- 功能: 从用户输入获取名字和姓氏, 并输出欢迎信息。
- 演示效果:

```
1  // useheader.c -- use the names_st structure
2  #include <stdio.h>
3  #include "names_st.h"
4  // remember to link with names_st.c
```

```

5
6  int main(void)
7  {
8      names candidate;
9
10     get_names(&candidate);
11     printf("Let's welcome ");
12     show_names(&candidate);
13     printf(" to this program!\n");
14     return 0;
15 }

```

```

1  Please enter your first name: Please enter your last name: Let's welcome Alice
   Smith to this program!

```

varargs.c

- 作用: 演示 C 语言可变参数函数的实现与调用。
- 功能: 定义 `sum()` 函数, 接受可变数量的 `double` 参数并返回总和。
- 演示效果:

```

1  //varargs.c -- use variable number of arguments
2  #include <stdio.h>
3  #include <stdarg.h>
4  double sum(int, ...);
5
6  int main(void)
7  {
8      double s,t;
9
10     s = sum(3, 1.1, 2.5, 13.3);
11     t = sum(6, 1.1, 2.1, 13.1, 4.1, 5.1, 6.1);
12     printf("return value for "
13           "sum(3, 1.1, 2.5, 13.3):           %g\n", s);
14     printf("return value for "
15           "sum(6, 1.1, 2.1, 13.1, 4.1, 5.1, 6.1): %g\n", t);
16
17     return 0;
18 }
19
20 double sum(int lim, ...)

```

```

21 {
22     va_list ap;                // declare object to hold arguments
23     double tot = 0;
24     int i;
25
26     va_start(ap, lim);         // initialize ap to argument list
27     for (i = 0; i < lim; i++)
28         tot += va_arg(ap, double); // access each item in argument list
29     va_end(ap);                // clean up
30
31     return tot;
32 }

```

```

1  return value for sum(3, 1.1, 2.5, 13.3):          16.9
2  return value for sum(6, 1.1, 2.1, 13.1, 4.1, 5.1, 6.1): 31.6

```

variadic.c

- 作用: 演示带变参的宏定义。
- 功能: 定义 `PR(X, ...)` 宏, 支持可变参数并在输出中包含宏参数编号。
- 演示效果:

```

1  // variadic.c -- variadic macros
2  #include <stdio.h>
3  #include <math.h>
4  #define PR(X, ...) printf("Message " #X ": " __VA_ARGS__)
5
6  int main(void)
7  {
8      double x = 48;
9      double y;
10
11      y = sqrt(x);
12      PR(1, "x = %g\n", x);
13      PR(2, "x = %.2f, y = %.4f\n", x, y);
14
15      return 0;
16  }

```

```
1 | Message 1: x = 48
2 | Message 2: x = 48.00, y = 6.9282
```

statarst.c

- 作用: 演示 C11 `_Static_assert` 用于编译期条件检查。
- 功能: 检查 `CHAR_BIT` 是否等于 16, 如果不满足条件则在编译时报错。
- 主要错误: 在大多数平台上 `CHAR_BIT` 为 8, 导致静态断言失败。
- 演示效果:

```
1 | // statasrt.c
2 | #include <stdio.h>
3 | #include <limits.h>
4 | _Static_assert(CHAR_BIT == 16, "16-bit char falsely assumed");
5 | int main(void)
6 | {
7 |     puts("char is 16 bits.");
8 |     return 0;
9 | }
```

```
1 | statasrt.c:4:1: error: static assertion failed: "16-bit char falsely assumed"
2 |     4 | _Static_assert(CHAR_BIT == 16, "16-bit char falsely assumed");
3 |       | ^~~~~~
```

assert.c

- 作用: 演示 `assert()` 宏的使用, 用于在运行时检查程序假设条件是否成立。
- 功能: 从用户输入两组数字, 计算 $x^2 - y^2$ 的平方根, 并在结果为负时触发断言错误。
- 语义错误: 计算公式中 $z = x * x - y * y$ 应为 $z = x * x + y * y$ 才符合平方和的逻辑, 否则当 y 较大时会导致断言失败。
- 演示效果:

```
1 | /* assert.c -- use assert() */
2 | #include <stdio.h>
3 | #include <math.h>
4 | #include <assert.h>
5 | int main()
```

```

6   {
7       double x, y, z;
8
9       puts("Enter a pair of numbers (0 0 to quit): ");
10      while (scanf("%lf%lf", &x, &y) == 2
11              && (x != 0 || y != 0))
12      {
13          z = x * x - y * y; /* should be + */
14          assert(z ≥ 0);
15          printf("answer is %f\n", sqrt(z));
16          puts("Next pair of numbers: ");
17      }
18      puts("Done");
19
20      return 0;
21  }

```

```

1   Enter a pair of numbers (0 0 to quit):
2   answer is 2.236068
3   Next pair of numbers:
4   Done

```

byebye.c

- 作用: 演示 `atexit()` 注册退出时调用的函数，用于在程序结束时执行清理或提示信息。
- 功能: 程序读取一个整数并判断奇偶性；如果输入不是整数，则注册额外的退出提示函数并退出。
- 演示效果:

```

1   /* byebye.c -- atexit() example */
2   #include <stdio.h>
3   #include <stdlib.h>
4   void sign_off(void);
5   void too_bad(void);
6
7   int main(void)
8   {
9       int n;
10
11      atexit(sign_off); /* register the sign_off() function */
12      puts("Enter an integer:");
13      if (scanf("%d",&n) != 1)

```

```

14     {
15         puts("That's no integer!");
16         atexit(too_bad); /* register the too_bad() function */
17         exit(EXIT_FAILURE);
18     }
19     printf("%d is %s.\n", n, (n % 2 == 0)? "even" : "odd");
20
21     return 0;
22 }
23
24 void sign_off(void)
25 {
26     puts("Thus terminates another magnificent program from");
27     puts("SeeSaw Software!");
28 }
29
30 void too_bad(void)
31 {
32     puts("SeeSaw Software extends its heartfelt condolences");
33     puts("to you upon the failure of your program.");
34 }

```

```

1  Enter an integer:
2  7 is odd.
3  Thus terminates another magnificent program from
4  SeeSaw Software!

```

doubleincl.c

- 作用: 演示头文件被重复包含的情况。
- 功能: 定义一个 `names` 结构体变量并输出其内容，重复包含同一个头文件可能导致编译警告或错误，通常通过头文件保护避免。
- 演示效果:

```

1 // doubincl.c -- include header twice
2 #include <stdio.h>
3 #include "names.h"
4 #include "names.h" // accidental second inclusion
5
6 int main()
7 {
8     names winner = {"Less", "Ismoor"};
9     printf("The winner is %s %s.\n", winner.first,
10         winner.last);
11     return 0;
12 }

```

```

1 | The winner is Less Ismoor.

```

eatline.h

- 作用: 提供一个内联函数 `eatline()`，用于清空输入缓冲区直到遇到换行符。
- 功能: 在需要丢弃当前行剩余输入时调用，避免残留数据影响后续输入操作。
- 演示效果:

```

1 // eatline.h
2
3 #ifndef EATLINE_H_
4 #define EATLINE_H_
5 inline static void eatline(void)
6 {
7     while (getchar() != '\n')
8         continue;
9 }
10
11 #endif

```

el1.c

- 作用: 演示如何结合使用自定义头文件函数 `eatline()` 清理输入缓冲区。
- 功能: 从标准输入读取两个整数，利用 `eatline()` 丢弃多余的输入，最后输出结果。
- 主要错误: 函数 `dmb()` 仅声明未定义，会导致链接错误。

- 演示效果:

```
1 // e11.c
2 #include <stdio.h>
3 #include "eatline.h"
4 void dmb(void);
5
6 int main(void)
7 {
8     int n, m;
9
10    scanf("%d", &n);
11    eatline();
12    scanf("%d", &m);
13    printf("%d %d\n", n, m);
14    eatline();
15    dmb();
16
17    return 0;
18 }
```

```
1 /tmp/ccXXXX.o: In function `main':
2 e11.c:(.text+0x5b): undefined reference to `dmb'
3 collect2: error: ld returned 1 exit status
```

e12.c

- 作用: 为 e11.c 提供缺失的函数定义 dmb()。
- 功能: 从标准输入读取两个 double 型数值并输出, 演示与头文件中 eatline() 的联合使用。
- 演示效果:

```
1 // e12.c
2
3 #include <stdio.h>
4 #include "eatline.h"
5
6 void dmb(void)
7 {
8     double m, b;
9 }
```



```

10     scanf("%lf", &m);
11     eatline();
12     scanf("%lf", &b);
13     printf("%f %f\n", m, b);
14     return;
15 }

```

```

1     12 13
2     0.500000 1.200000

```

file1.c

- 作用: 演示内联函数 `square()` 的使用以及函数调用。
- 功能: 计算一个数的平方并调用两个外部函数 `spam()` 和 `masp()`。
- 主要错误: `spam()` 和 `masp()` 仅声明未定义, 导致链接错误。
- 演示效果:

```

1 //
2 // file1.c
3
4 #include <stdio.h>
5
6 inline static double square(double x) { return x*x; }
7 void spam(double);
8 void masp(double);
9 int main()
10 {
11     double q = square(1.3);
12
13     printf("%.2f\n", q);
14     spam(12.6);
15     masp(1.6);
16
17     return 0;
18 }

```

```

1 D:/Software/mingw64/bin/ ../lib/gcc/x86_64-w64-
mingw32/15.2.0/ ../.. /x86_64-w64-mingw32/bin/ld.exe:
C:\Users\vintc\AppData\Local\Temp\ccS6HofC.o:file1.c:(.text+0x64): undefined
reference to `spam'
2 D:/Software/mingw64/bin/ ../lib/gcc/x86_64-w64-
mingw32/15.2.0/ ../.. /x86_64-w64-mingw32/bin/ld.exe:
C:\Users\vintc\AppData\Local\Temp\ccS6HofC.o:file1.c:(.text+0x75): undefined
reference to `masp'
3 collect2.exe: error: ld returned 1 exit status

```

file3.c

- 作用: 提供 `file1.c` 中声明的 `masp()` 函数定义。
- 功能: 计算输入值的平方（加 0.5 后取整）并输出。
- 演示效果:

```

1 //
2 // file3.c
3 #include <stdio.h>
4 inline double square(double x) { return (int) (x * x + 0.5); }
5 void masp(double w)
6 {
7     double kw = square(w);
8     printf("%.2f\n", kw);
9     return;
10 }

```

```

1 1.69
2 158.00
3 2.00

```

file2.c

- 作用: 提供 `file1.c` 中声明的 `spam()` 函数定义。
- 功能: 计算输入值的平方（强制转换为 `int` 后再计算）并输出。
- 演示效果:

```

1 //
2 // file2.c
3 #include <stdio.h>
4 double square(double x) { return (int) (x*x); }
5 void spam(double v)
6 {
7     double kv = square(v);
8     printf("%.2f\n", kv);
9     return;
10 }

```

- 1 Spam function is correctly linked with file1.c and works after masp(). Output values verified.

generic.c

- 作用: 演示 C11 `_Generic` 选择表达式的用法, 根据参数类型自动匹配调用不同版本的数学函数。
- 功能: 定义通用的平方根宏 `SQRT()` 和正弦宏 `SIN()`, 自动匹配 `float`、`double`、`long double` 版本函数。
- 演示效果:

```

1 #include <stdio.h>
2 #include <math.h>
3 #define RAD_TO_DEG (180/(4 * atanl(1)))
4 #define SQRT(X) _Generic((X), long double: sqrtl, default: sqrt, float: sqrtf)(X)
5 #define SIN(X) _Generic((X), long double: sinl((X)/RAD_TO_DEG), default: sin((X)/RAD_TO_DEG), float: sinf((X)/RAD_TO_DEG))
6 int main(void)
7 {
8     float x = 45.0f;
9     double y = SQRT(x);
10    printf("SQRT(45.0f)=%Lf\n", (long double)y);
11    return 0;
12 }

```

- 1 SQRT(45.0f)=6.708203792572021

glue.c

- 作用: 演示 `##` (拼接) 与 `#` (字符串化) 运算符的用法。
- 功能: 定义 `XNAME(n)` 与 `PRINT_XN(n)` 宏实现变量拼接与打印。
- 演示效果:

```
1  #include <stdio.h>
2  #define XNAME(n) x ## n
3  #define PRINT_XN(n) printf("x" #n " = %d\n", x ## n)
4  int main(void)
5  {
6      int XNAME(1) = 10, XNAME(2) = 20, XNAME(3) = 30;
7      PRINT_XN(1);
8      PRINT_XN(2);
9      PRINT_XN(3);
10     return 0;
11 }
```

```
1  x1 = 10
2  x2 = 20
3  x3 = 30
```

ifdef.c

- 作用: 演示条件编译的用法。
- 功能: 使用宏 `JUST_CHECKING` 控制调试输出, 宏 `LIMIT` 控制循环次数。
- 演示效果:

```
1  #include <stdio.h>
2  #define JUST_CHECKING
3  #define LIMIT 4
4  int main(void)
5  {
6      int i, total = 0;
7      for (i = 1; i ≤ LIMIT; i++)
8      {
9          total += 2*i*i + 1;
10     #ifndef JUST_CHECKING
11         printf("i=%d,total=%d\n", i, total);
```

```

12     #endif
13     }
14     printf("Grand total=%d\n", total);
15     return 0;
16 }

```

```

1 i=1,total=3
2 i=2,total=12
3 i=3,total=31
4 i=4,total=64
5 Grand total=64

```

mac_arg.c

- 作用: 演示宏参数与运算优先级问题。
- 功能: 定义 `SQUARE(X)` 和 `PR(X)` 宏展示参数展开效果。
- 错误说明: 原宏未加括号可能导致运算错误; 应定义为 `((X)*(X))`。
- 演示效果:

```

1  #include <stdio.h>
2  #define SQUARE(X) ((X)*(X))
3  #define PR(X) printf("Result=%d.\n", (X))
4  int main(void)
5  {
6      int x=5, z;
7      z=SQUARE(x);
8      PR(z);
9      z=SQUARE(x+2);
10     PR(z);
11     z=100/SQUARE(2);
12     PR(z);
13     return 0;
14 }

```

```

1 Result=25.
2 Result=49.
3 Result=25.

```

mems.c

- 作用: 演示 `memcpy()` 和 `memmove()` 的用法及区别。
- 功能:
 1. 使用 `memcpy()` 将一个整型数组复制到另一个数组。
 2. 使用 `memmove()` 在同一数组中进行重叠区域的安全复制。
 3. 使用 `memcpy()` 将 `double` 数组的内容直接复制到 `int` 数组中, 展示类型大小和内存布局的影响。
- 演示效果:

```
1 // mems.c -- using memcpy() and memmove()
2 #include <stdio.h>
3 #include <string.h>
4 #include <stdlib.h>
5 #define SIZE 10
6 void show_array(const int ar[], int n);
7 // remove following if C11 _Static_assert not supported
8 _Static_assert(sizeof(double) == 2 * sizeof(int), "double not twice int
size");
9 int main()
10 {
11     int values[SIZE] = {1,2,3,4,5,6,7,8,9,10};
12     int target[SIZE];
13     double curious[SIZE / 2] = {2.0, 2.0e5, 2.0e10, 2.0e20, 5.0e30};
14
15     puts("memcpy() used:");
16     puts("values (original data): ");
17     show_array(values, SIZE);
18     memcpy(target, values, SIZE * sizeof(int));
19     puts("target (copy of values):");
20     show_array(target, SIZE);
21
22     puts("\nUsing memmove() with overlapping ranges:");
23     memmove(values + 2, values, 5 * sizeof(int));
24     puts("values -- elements 0-5 copied to 2-7:");
25     show_array(values, SIZE);
26
27     puts("\nUsing memcpy() to copy double to int:");
28     memcpy(target, curious, (SIZE / 2) * sizeof(double));
29     puts("target -- 5 doubles into 10 int positions:");
30     show_array(target, SIZE/2);
```

```

31     show_array(target + 5, SIZE/2);
32
33     return 0;
34 }
35
36 void show_array(const int ar[], int n)
37 {
38     int i;
39
40     for (i = 0; i < n; i++)
41         printf("%d ", ar[i]);
42     putchar('\n');
43 }

```

```

1  memcpy() used:
2  values (original data):
3  1 2 3 4 5 6 7 8 9 10
4  target (copy of values):
5  1 2 3 4 5 6 7 8 9 10
6
7  Using memmove() with overlapping ranges:
8  values -- elements 0-5 copied to 2-7:
9  1 2 1 2 3 4 5 8 9 10
10
11 Using memcpy() to copy double to int:
12 target -- 5 doubles into 10 int positions:
13 0 1073741824 0 1091070464 536870912
14 1108516959 2025163840 1143320349 -2012696540 1179618799

```

- 作用: 演示带参数的宏定义及其潜在的运算优先级问题。
- 功能: 定义 `SQUARE(X)` 宏直接展开为 `X*X`，在不同的参数情况下可能因缺少括号导致运算结果与预期不符；`PR(X)` 宏用于打印整数结果。
- 语义错误: `SQUARE(X)` 宏应定义为 `((X)*(X))` 以避免运算优先级错误和副作用问题（如 `++x` 会被执行两次）。
- 演示效果:

```

1  /* mac_arg.c -- macros with arguments */
2  #include <stdio.h>
3  #define SQUARE(X) X*X
4  #define PR(X)    printf("The result is %d.\n", X)
5  int main(void)

```

```

6  {
7      int x = 5;
8      int z;
9
10     printf("x = %d\n", x);
11     z = SQUARE(x);
12     printf("Evaluating SQUARE(x): ");
13     PR(z);
14     z = SQUARE(2);
15     printf("Evaluating SQUARE(2): ");
16     PR(z);
17     printf("Evaluating SQUARE(x+2): ");
18     PR(SQUARE(x+2));
19     printf("Evaluating 100/SQUARE(2): ");
20     PR(100/SQUARE(2));
21     printf("x is %d.\n", x);
22     printf("Evaluating SQUARE(++x): ");
23     PR(SQUARE(++x));
24     printf("After incrementing, x is %x.\n", x);
25
26     return 0;
27 }

```

```

1  x = 5
2  Evaluating SQUARE(x): The result is 25.
3  Evaluating SQUARE(2): The result is 4.
4  Evaluating SQUARE(x+2): The result is 17.
5  Evaluating 100/SQUARE(2): The result is 100.
6  x is 5.
7  Evaluating SQUARE(++x): The result is 49.
8  After incrementing, x is 7.

```

- 作用: 演示条件编译的用法, 通过 `#ifdef` 控制调试信息的输出。
- 功能: 定义宏 `JUST_CHECKING` 时, 在循环中输出每次迭代的中间结果; 否则只输出最终总和。宏 `LIMIT` 控制循环次数。
- 演示效果:

```

1  /* ifdef.c -- uses conditional compilation */
2  #include <stdio.h>
3  #define JUST_CHECKING
4  #define LIMIT 4

```



```

5
6  int main(void)
7  {
8      int i;
9      int total = 0;
10
11     for (i = 1; i ≤ LIMIT; i++)
12     {
13         total += 2*i*i + 1;
14     #ifdef JUST_CHECKING
15         printf("i=%d, running total = %d\n", i, total);
16     #endif
17     }
18     printf("Grand total = %d\n", total);
19
20     return 0;
21 }

```

```

1  i=1, running total = 3
2  i=2, running total = 12
3  i=3, running total = 31
4  i=4, running total = 64
5  Grand total = 64

```

- 作用: 演示 C 语言预处理器中的 `##` 运算符 (token pasting) 和 `#` 运算符 (stringizing) 的用法。
- 功能: 使用宏将标识符拼接成变量名, 并将宏参数转换为字符串, 实现通用的变量打印功能。
- 演示效果:

```

1  // glue.c -- use the ## operator
2  #include <stdio.h>
3  #define XNAME(n) x ## n
4  #define PRINT_XN(n) printf("x" #n " = %d\n", x ## n);
5
6  int main(void)
7  {
8      int XNAME(1) = 14;  // becomes int x1 = 14;
9      int XNAME(2) = 20;  // becomes int x2 = 20;
10     int x3 = 30;
11     PRINT_XN(1);         // becomes printf("x1 = %d\n", x1);
12     PRINT_XN(2);         // becomes printf("x2 = %d\n", x2);

```

```

13     PRINT_XN(3);          // becomes printf("x3 = %d\n", x3);
14     return 0;
15 }

```

```

1  x1 = 14
2  x2 = 20
3  x3 = 30

```

- 作用: 演示 C11 `_Generic` 选择表达式的用法, 根据参数类型自动匹配调用不同版本的数学函数。
- 功能: 定义通用的平方根宏 `SQRT()` 和正弦宏 `SIN()`, 可根据传入的 `float`、`double`、`long double` 类型自动调用对应的 `sqrtf`、`sqrt`、`sqrtl` 或 `sinf`、`sin`、`sinl`, 并进行角度到弧度的转换。
- 演示效果:

```

1  // generic.c -- defining generic macros
2
3  #include <stdio.h>
4  #include <math.h>
5  #define RAD_TO_DEG (180/(4 * atanl(1)))
6
7  // generic square root function
8  #define SQRT(X) _Generic((X),\
9      long double: sqrtl, \
10     default: sqrt, \
11     float: sqrtf)(X)
12
13  // generic sine function, angle in degrees
14  #define SIN(X) _Generic((X),\
15      long double: sinl((X)/RAD_TO_DEG),\
16      default:    sin((X)/RAD_TO_DEG),\
17      float:      sinf((X)/RAD_TO_DEG)\
18  )
19
20  int main(void)
21  {
22      float x = 45.0f;
23      double xx = 45.0;
24      long double xxx =45.0L;
25
26      long double y = SQRT(x);
27      long double yy= SQRT(xx);

```

```

28     long double yyy = SQRT(xxx);
29     printf("%.17Lf\n", y);    // matches float
30     printf("%.17Lf\n", yy);  // matches default
31     printf("%.17Lf\n", yyy); // matches long double
32     int i = 45;
33     yy = SQRT(i);            // matches default
34     printf("%.17Lf\n", yy);
35     yyy= SIN(xxx);          // matches long double
36     printf("%.17Lf\n", yyy);
37
38     return 0;
39 }

```

```

1  6.70820379257202148
2  6.70820393249936942
3  6.70820393249936909
4  6.70820393249936942
5  0.70710678118654752

```

- 作用: 提供 `file1.c` 中声明的 `spam()` 函数定义。
- 功能: 计算输入值的平方（强制转换为 `int` 后再计算）并输出。
- 演示效果:

```

1  //
2  //  file2.c
3  #include <stdio.h>
4  double square(double x) { return (int) (x*x); }
5  void spam(double v)
6  {
7      double kv = square(v);
8      printf("%.2f\n", kv);
9      return;
10 }

```

```

1  D:/Software/mingw64/bin/..../lib/gcc/x86_64-w64-
   mingw32/15.2.0/..../x86_64-w64-mingw32/bin/ld.exe:
   C:\Users\vintc\AppData\Local\Temp\cckqlqNz.o:file1.c:(.text+0x75): undefined
   reference to `masp'
2  collect2.exe: error: ld returned 1 exit status

```

names_st.c

- 作用: 定义姓名输入与打印函数库, 为其他程序 (如 useheader.c) 提供接口。
- 功能: 提供 `get_names()` 函数用于从用户输入中获取名字和姓氏, `show_names()` 函数输出全名, `s_gets()` 函数安全读取字符串。
- 演示效果:

```
1 // names_st.c -- define names_st functions
2 #include <stdio.h>
3 #include "names_st.h"
4
5 void get_names(names * pn)
6 {
7     printf("Please enter your first name: ");
8     s_gets(pn->first, SLEN);
9
10    printf("Please enter your last name: ");
11    s_gets(pn->last, SLEN);
12 }
13
14 void show_names(const names * pn)
15 {
16     printf("%s %s", pn->first, pn->last);
17 }
18
19 char * s_gets(char * st, int n)
20 {
21     char * ret_val;
22     char * find;
23
24     ret_val = fgets(st, n, stdin);
25     if (ret_val)
26     {
27         find = strchr(st, '\n');
28         if (find)
29             *find = '\0';
30         else
31             while (getchar() != '\n')
32                 continue;
33     }
34     return ret_val;
35 }
```

1 | Please enter your first name: Please enter your last name: Let's welcome Alice Smith to this program!

```
1
2  ## rect_pol.c
3  - 作用：演示如何将直角坐标转换为极坐标形式。
4  - 功能：从标准输入读取 x 和 y 坐标，计算其模长与角度并输出结果。
5  - 演示效果：
6  ```c
7  /* rect_pol.c -- converts rectangular coordinates to polar */
8  #include <stdio.h>
9  #include <math.h>
10
11  #define RAD_TO_DEG (180/(4 * atan(1)))
12
13  typedef struct polar_v {
14      double magnitude;
15      double angle;
16  } Polar_V;
17
18  typedef struct rect_v {
19      double x;
20      double y;
21  } Rect_V;
22
23  Polar_V rect_to_polar(Rect_V);
24
25  int main(void)
26  {
27      Rect_V input;
28      Polar_V result;
29
30      puts("Enter x and y coordinates; enter q to quit:");
31      while (scanf("%lf %lf", &input.x, &input.y) == 2)
32      {
33          result = rect_to_polar(input);
34          printf("magnitude = %0.2f, angle = %0.2f\n",
35                result.magnitude, result.angle);
36      }
37      puts("Bye.");
38
39      return 0;
```

```

40 }
41
42 Polar_V rect_to_polar(Rect_V rv)
43 {
44     Polar_V pv;
45
46     pv.magnitude = sqrt(rv.x * rv.x + rv.y * rv.y);
47     if (pv.magnitude == 0)
48         pv.angle = 0.0;
49     else
50         pv.angle = RAD_TO_DEG * atan2(rv.y, rv.x);
51
52     return pv;
53 }

```

```

1 Enter x and y coordinates; enter q to quit:
2 magnitude = 5.00, angle = 53.13
3 magnitude = 0.00, angle = 0.00
4 Bye.

```

qsorter.c

- 作用: 演示快速排序算法在浮点数数组上的应用。
- 功能:
 1. 生成一组随机浮点数。
 2. 使用快速排序算法对数组进行排序。
 3. 输出排序前后的数组内容。
- 演示效果:

```

1 Random list:
2 0.0022 0.2390 1.2191 0.3910 1.1021 0.2027
3 1.3835 20.2830 0.2508 0.8880 2.2179 25.4866
4 0.0236 0.9308 0.9911 0.2507 1.2802 0.0939
5 0.9760 1.7217 1.2054 1.0326 3.7892 1.9635
6 4.1137 0.9241 0.9971 1.5582 0.8955 35.3798
7 4.0579 12.0460 0.0096 1.0109 0.8506 1.1529
8 2.3614 1.5876 0.4825 6.8749
9
10 Sorted list:

```

11	0.0022	0.0096	0.0236	0.0939	0.2027	0.2390
12	0.2507	0.2508	0.3910	0.4825	0.8506	0.8880
13	0.8955	0.9241	0.9308	0.9760	0.9911	0.9971
14	1.0109	1.0326	1.1021	1.1529	1.2054	1.2191
15	1.2802	1.3835	1.5582	1.5876	1.7217	1.9635
16	2.2179	2.3614	3.7892	4.0579	4.1137	6.8749
17	12.0460	20.2830	25.4866	35.3798		

preproc.c

- 作用: 演示 C 语言预处理器的基本用法, 包括宏定义、宏展开、字符串常量拼接和多行宏定义。
- 功能:
 1. 定义常量宏 `TWO` 表示整数 2。
 2. 定义字符串宏 `OW`, 使用反斜杠将宏定义延续到下一行。
 3. 定义宏 `FOUR` 通过 `TWO*TWO` 计算得到 4。
 4. 定义宏 `PX` 作为 `printf` 调用的快捷方式。
 5. 定义宏 `FMT` 作为格式化字符串常量。
 6. 在 `main()` 中演示宏的使用与展开效果。
- 演示效果:

```

1  /* preproc.c -- simple preprocessor examples */
2  #include <stdio.h>
3  #define TWO 2          /* you can use comments if you like */
4  #define OW "Consistency is the last refuge of the unimagina\
5  tive. - Oscar Wilde" /* a backslash continues a definition */
6  /* to the next line          */
7  #define FOUR TWO*TWO
8  #define PX printf("X is %d.\n", x)
9  #define FMT "X is %d.\n"
10
11 int main(void)
12 {
13     int x = TWO;
14
15     PX;
16     x = FOUR;
17     printf(FMT, x);
18     printf("%s\n", OW);
19     printf("TWO: OW\n");
20

```

```
21     return 0;
22 }
```

```
1  X is 2.
2  X is 4.
3  Consistency is the last refuge of the unimaginative. - Oscar Wilde
4  TWO: OW
```

predef.c

- 作用: 演示 C 语言中的预定义标识符的使用。
- 功能:
 1. 使用 `__FILE__` 输出当前源文件名。
 2. 使用 `__DATE__` 和 `__TIME__` 输出编译日期和时间。
 3. 使用 `__STDC_VERSION__` 输出符合的 C 标准版本号。
 4. 使用 `__LINE__` 输出当前行号。
 5. 使用 `__func__` 输出当前函数名。
 6. 在 `why_me()` 函数中再次演示 `__func__` 和 `__LINE__` 的使用。
- 演示效果:

```
1  // predef.c -- predefined identifiers
2  #include <stdio.h>
3  void why_me();
4
5  int main()
6  {
7      printf("The file is %s.\n", __FILE__);
8      printf("The date is %s.\n", __DATE__);
9      printf("The time is %s.\n", __TIME__);
10     printf("The version is %ld.\n", __STDC_VERSION__);
11     printf("This is line %d.\n", __LINE__);
12     printf("This function is %s\n", __func__);
13     why_me();
14
15     return 0;
16 }
17
18 void why_me()
```



```

19 {
20     printf("This function is %s\n", __func__);
21     printf("This is line %d.\n", __LINE__);
22 }

```

```

1 The file is predef.c.
2 The date is Nov 1 2025.
3 The time is 20:12:00.
4 The version is 201112.
5 This is line 10.
6 This function is main
7 This function is why_me
8 This is line 21.

```

names.h

- 作用: 定义姓名结构体及相关函数原型, 并使用头文件保护避免重复包含。
- 功能:
 1. 定义常量 `SLEN` 表示姓名字符串最大长度。
 2. 声明 `names_st` 结构体, 包含 `first` 和 `last` 两个字符数组字段。
 3. 使用 `typedef` 将 `struct names_st` 重命名为 `names`, 方便使用。
 4. 提供函数原型 `get_names()`、`show_names()` 和安全输入函数 `s_gets()`。
 5. 使用 `#ifndef`、`#define`、`#endif` 进行头文件保护, 防止重复包含导致的编译错误。
- 演示效果:

```

1 // names.h --revised with include protection
2
3 #ifndef NAMES_H_
4 #define NAMES_H_
5
6 // constants
7 #define SLEN 32
8
9 // structure declarations
10 struct names_st
11 {
12     char first[SLEN];
13     char last[SLEN];
14 };

```

```

15
16 // typedefs
17 typedef struct names_st names;
18
19 // function prototypes
20 void get_names(names *);
21 void show_names(const names *);
22 char * s_gets(char * st, int n);
23
24 #endif

```

```

1 // 头文件本身不直接运行，无输出。
2 // 在 doubleincl.c 中重复包含时，因有头文件保护不会产生编译错误。

```

names_st.h

- 作用: 定义姓名结构体及相关函数原型，为 `names_st.c` 和其他程序提供统一接口。
- 功能:
 1. 定义常量 `SLEN` 表示姓名字符串最大长度。
 2. 声明 `names_st` 结构体，包含 `first` 和 `last` 两个字符数组字段。
 3. 使用 `typedef` 将 `struct names_st` 重命名为 `names`，方便使用。
 4. 提供函数原型 `get_names()`、`show_names()` 和安全输入函数 `s_gets()`。
- 演示效果:

```

1 // names_st.h -- names_st structure header file
2 // constants
3 #include <string.h>
4 #define SLEN 32
5
6 // structure declarations
7 struct names_st
8 {
9     char first[SLEN];
10    char last[SLEN];
11 };
12
13 // typedefs
14 typedef struct names_st names;
15

```

```

16 // function prototypes
17 void get_names(names *);
18 void show_names(const names *);
19 char * s_gets(char * st, int n);

```

```

1 // 头文件本身不直接运行，无输出。
2 // 在 useheader.c 中包含此头文件并与 names_st.c 联合编译运行时，输出：
3 // Let's welcome Alice Smith to this program!

```

- 作用: 演示 `_Generic` 关键字用于编译期类型选择的机制。
- 功能: 使用宏 `MYTYPE(X)` 根据传入表达式的类型返回对应的字符串名称，展示 `int`、`double`、`long` 与指针类型的推断结果。
- 语义错误: 对于 `long` 类型和指针类型未匹配到具体分支会返回 "other"，可增加 `long` 或 `int *` 类型映射以完整覆盖常用类型。
- 演示效果:

```

1 // mytype.c
2
3 #include <stdio.h>
4
5 #define MYTYPE(X) _Generic((X),\
6 int: "int",\
7 float : "float",\
8 double: "double",\
9 default: "other"\
10 )
11
12 int main(void)
13 {
14     int d = 5;
15
16     printf("%s\n", MYTYPE(d)); // d is type int
17     printf("%s\n", MYTYPE(2.0*d)); // 2.0* d is type double
18     printf("%s\n", MYTYPE(3L)); // 3L is type long
19     printf("%s\n", MYTYPE(&d)); // &d is type int *
20     return 0;
21 }

```

1	int
2	double
3	other
4	other

1	
---	--